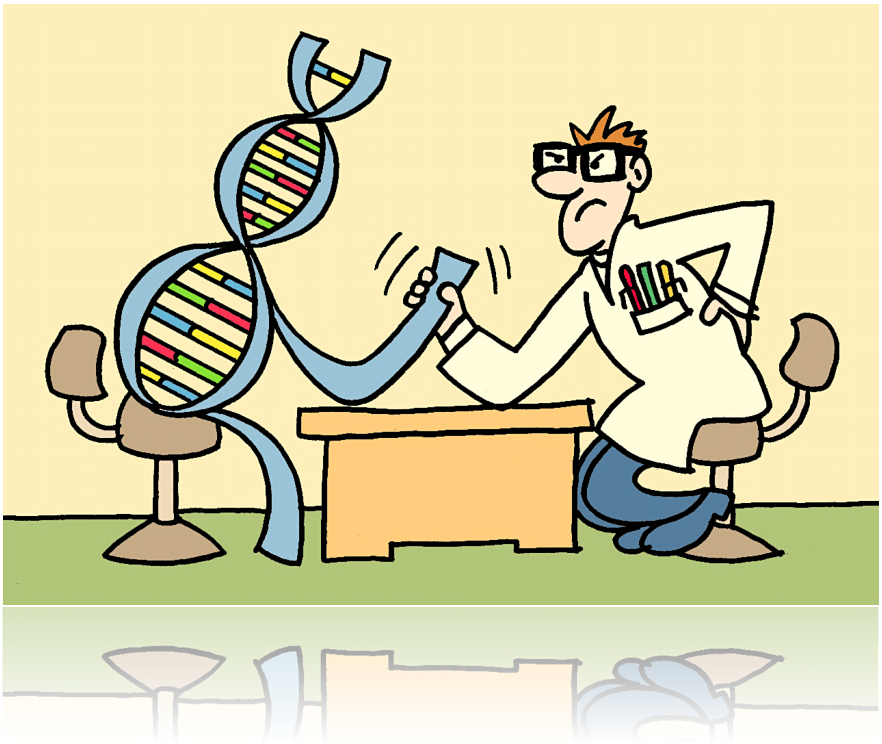
Computational Intelligence II: Evolutionary Optimisation

Chapter 5: Genetic Programming

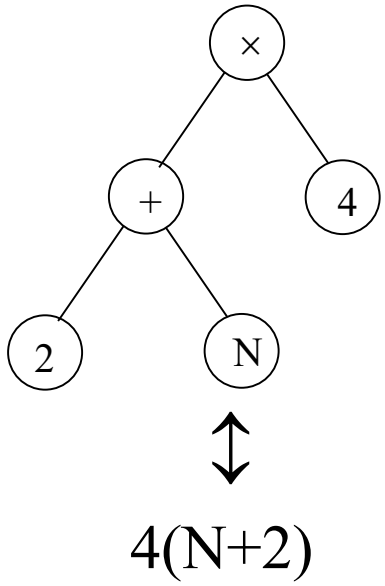


Genetic programming (GP)

⌘ Genetic Algorithms were previously applied to finding the optimal design (optimal values of parameters) of a given system; that is the an optimal instance or parameterisation of a model of a fixed form.

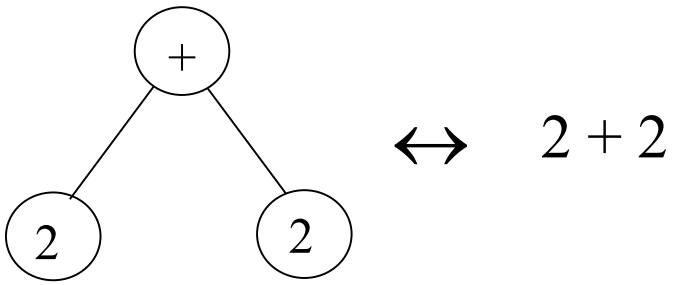
⌘ There is no reason why the same type of approach cannot be applied to additionally find the optimal form of a model or system (e.g., a computer program, or a set of mathematical formulae) !!

- ◎ Instead of varying the parameters alone, the genes also encode a set of basic functions that can be applied to an input.
- ◎ The main difference between GAs and GP is that the gene contains functions as well as variables/numbers. However there are fundamental differences in the structure of the gene.
- ◎ Genetic programming as pioneered by Koza (*Genetic Programming: On the programming of computers by Natural Selection*, 1992), uses tree structures, where the branching nodes are functions and leaf nodes are constants or input variables.
- ◎ LISP and PROLOG function sets were initially used, but any programming language is possible, such as C++ or Matlab.



Terminal sets

- The terminal set T is the set of arguments for the functions in the genetic program. They correspond to the **leaves** on the tree structure and, therefore, terminate branches.
- A genetic program is a combination of branches of functions, each branch ending with a leaf that contains an argument.
  - Example:



- The argument could be a **fixed number** or an **external input** (i.e., variable) from another system or from data set.
- Note, that most of the simple functions considered have two leaves/arguments (binary functions), but this is not necessary. Some functions could have one (unary), three (ternary) or more inputs.

Functions sets

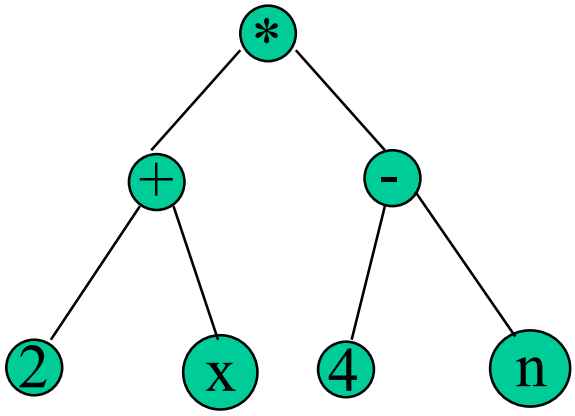
- The function set F is a collection of the basic building blocks of the algorithm. The set should be small so that the search space is relatively small, but big enough to provide flexibility and expressive power to cover the solution space and computability requirements.

- These functions typically include:
  - Logical operations: and, or, xor, not.
  - Simple arithmetic functions: +, -, *, % (protected division), abs (absolute value), sign, etc.
  - Advanced mathematical functions: sin, cos, tan, etc.
  - Conditional execution: if/then/else, etc.
  - Repetitive execution: for & while loops, vector/matrix arithmetic, etc.
  - Bespoke functions.

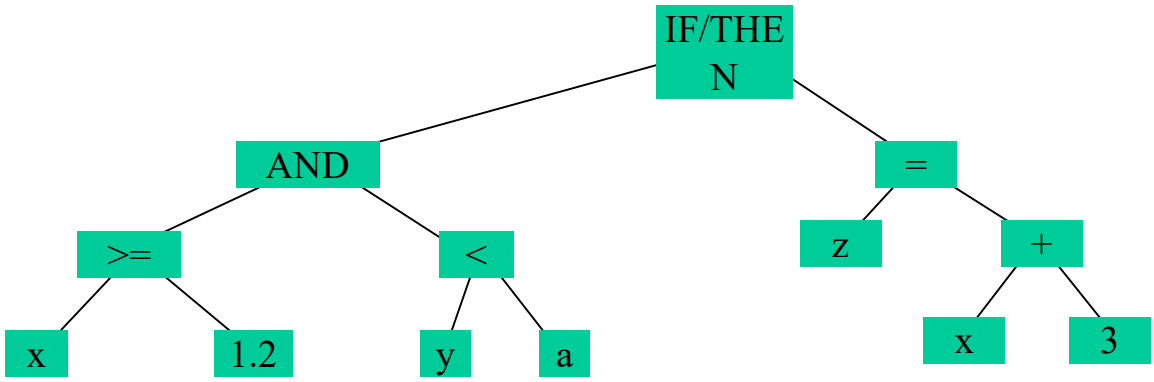
- As with standard genetic coding, the eventual performance of the final solution/algorithm can be affected dramatically by a good or bad coding (choice of functions).

Examples

⌘ f(x;n) = (2 + x)*(4 - n)



⌘ IF (x >= 1.2) AND (y<a) THEN z=x+3



Examples - applications

⌘ **Symbolic Regression**: the process of discovering both the functional form of a target (but unknown) function and all of its parameters/coefficients, given sample data.

Example:
Given n points in R^2, $(x_1, y_1), \ldots, (x_n, y_n)$,
find the function f(x) s.t. $f(x_i) = y_i$, for all i = 1, …, n
Function set F = {+, -, *, /, sin, cos}
Terminal Set $T = R \cup \{x\}$
The objective function is the error: $\sum_{i=1}^{n} \left| f(x_i) - y_i \right|$

⌘ **Analog circuit design**: An initial circuit is modified to create a new circuit according to functionality criteria.

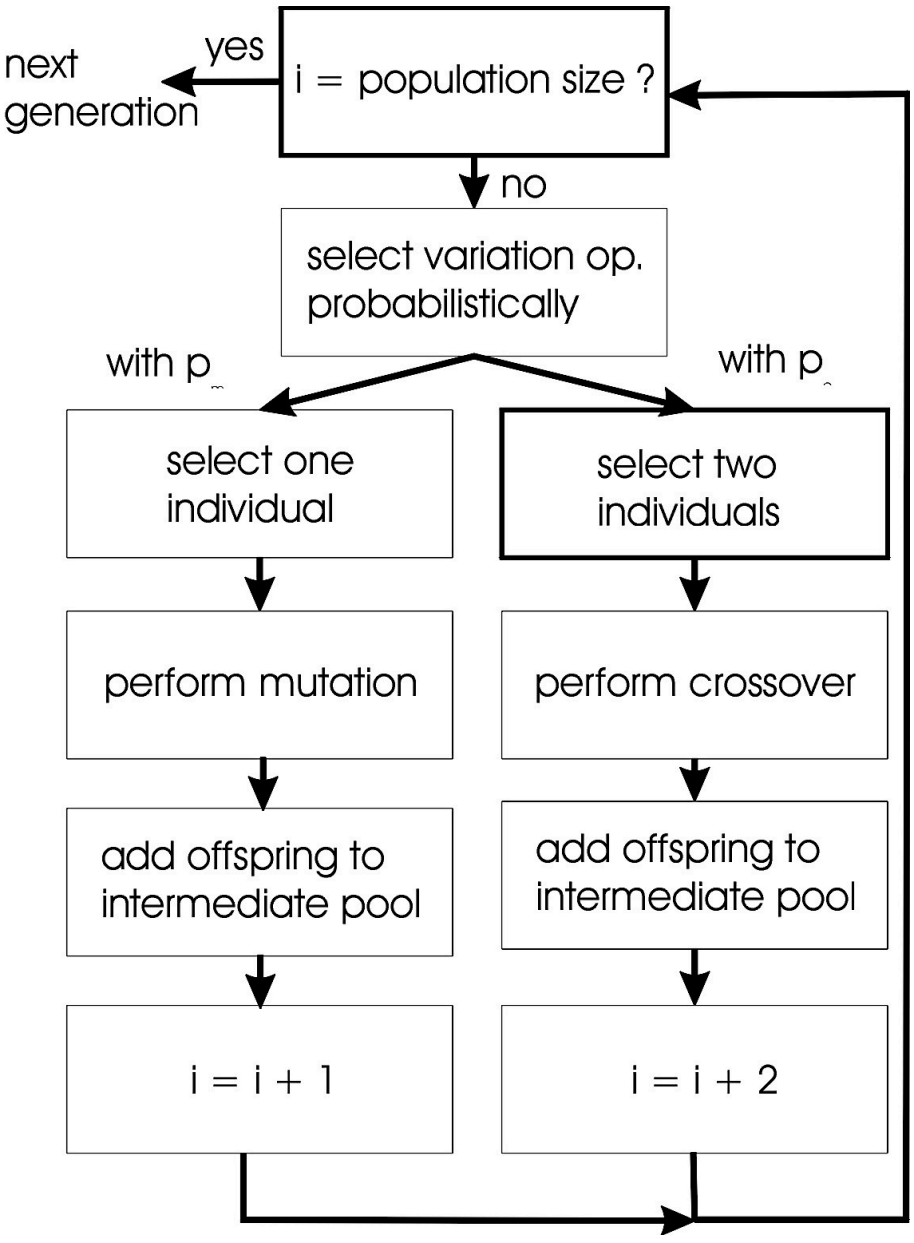
⌘ **Image processing**: Construction of image processing functions or filters from logical operations on the images (AND, OR, etc.) rather than using mathematical functions.

- ◎ The aim could be to enhance certain features within images by training the filters on a set of example images.
- ◎ The logical operations are simple to implement in hardware, and the combination of logical operations can produce operations that would not necessarily have been derived using a disciplined mathematical approach.

GP operators

- GP uses the same basic genetic processes as the GAs, although their implementation may be very different.
- The population now consists of a number of trial computer programs!
- **Selection**: As before, a trial program from the current population is selected to survive or mate based on its relative fitness value.
- **Crossover**: As before, pairs of solutions are selected form the current population to produce children. A node is selected independently in each parent and the branch (or leaf) is swapped at those nodes.
- **Mutation**: As before, mutation can change values of the leaves or the functions at the branching nodes.
- In GP, due to the complexity of the child solutions (computer programs) crossover and mutation are applied independently (applied separately to existing population parent programs).

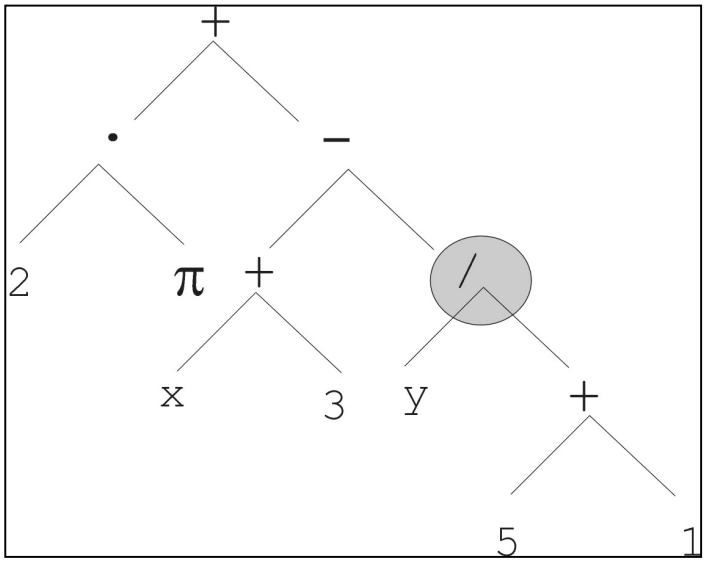
GP flowchart



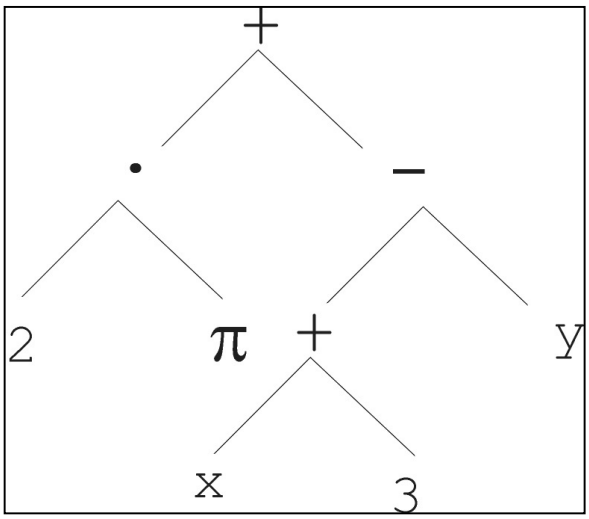
Example - mutation

⌘ Replace randomly chosen subtree by randomly generated tree:

Parent:

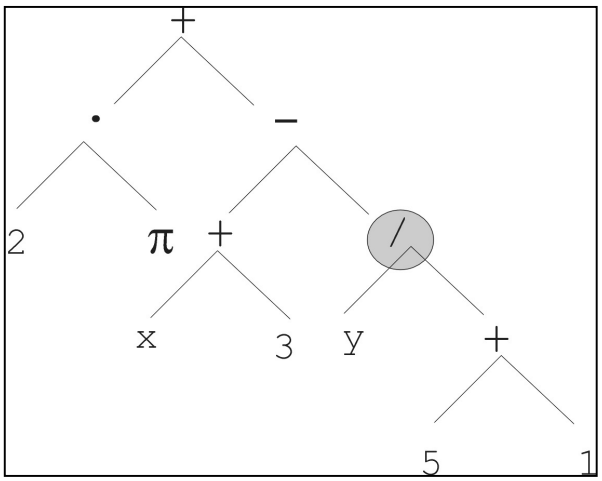


Child:

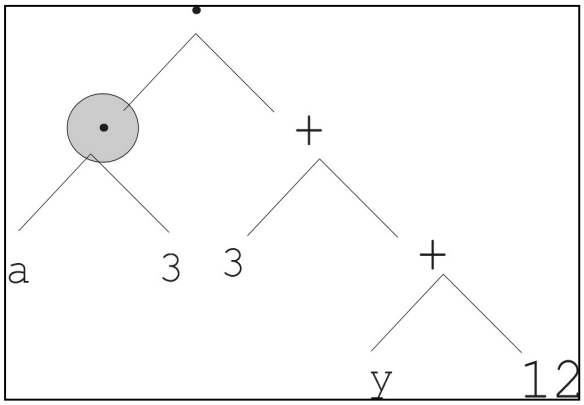


Example - crossover

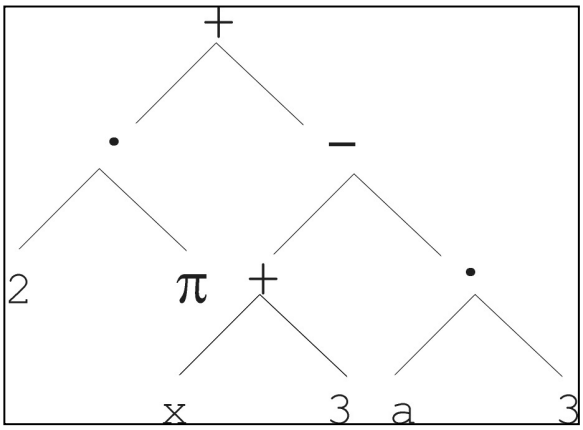
⌘ Exchange two randomly chosen subtrees among the parents:



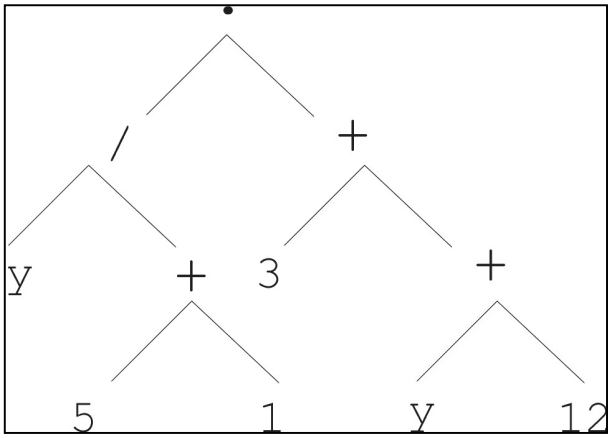
parent 1



parent 2



child 1



child 2

Population initialisation

- Maximum initial depth D_{max} of trees is set.

- **Full method** (each branch has depth = D_{max}):
  - Nodes at depth $d < D_{max}$ are randomly chosen from function set F.
  - Nodes at depth $d = D_{max}$ are randomly chosen from terminal set T.

- **Grow method** (each branch has depth $\leq D_{max}$):
  - Nodes at depth $d < D_{max}$ are randomly chosen from $F \cup T$.
  - Nodes at depth $d = D_{max}$ are randomly chosen from T.

- **Ramped half-and-half**: A common GP initialisation where the Grow and Full methods each deliver half of initial population.

Introns

- Another aspect of evolutionary programming is the creation of structures within a program that do nothing.
  - The information exists in the genes (it is part of the genotype) but it plays no part in the behaviour of the system (it is not essential part of the phenotype).
  - For example, intron is an "if statement" that contains a large amount of code, but will never be executed because the associated condition can never be satisfied.
- This type of genetic structure is common in biological systems and is referred to as an **intron**.
- Although introns do not have any effect on the operation of the genetic program, they do have some useful but also undesirable characteristics.
  - Toward the end of the optimisation process, when most of the population has converged, the presence of introns in the genetic program tends to protect the solutions against destructive crossover operations.
  - However, the problem is that introns increase the complexity of the solution and require more computational resources (execution time and memory).

Complexity & introns

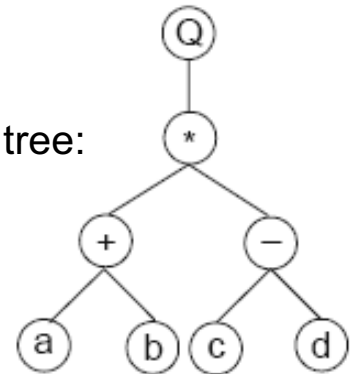
- One way of avoiding introns is to penalise longer programs.
- To achieve this, it is necessary to define some sort of measure that can be used to assess the complexity or "length" of a computer program.
- There are many different measures that have been proposed to define complexity:
  - Number of nodes.
  - Number of instructions.
  - Number of CPU instructions required to express the program.

Gene Expression Programming

- GEP is inspired from Open Reading Frame (**ORF**) coding in biology:
  - [start codon - amino acid codons - termination codon].
  - It is also referred to as the **K-expression**.
- It is similar to GP, but uses a different encoding scheme:
  - For example assume the algebraic expression:

$$\sqrt{(a+b)\times(c-d)}$$

  - This can be encoded by the tree:



  - The above two phenotypic expressions can produce (be encoded to) the genotype:

```
Q * + - a b c d
```

  - Starting from "Q (square root)" (position 1) and ending with terminal "d" (position 8).
  - This differs from pre/post-fix notations used in programming.

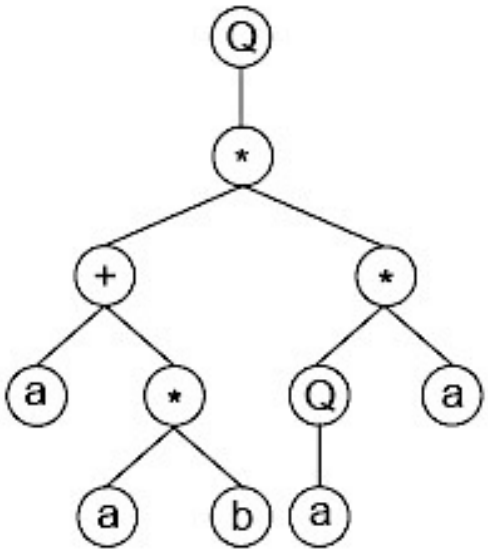
GEP decoding

⌘ The mapping from genotype to phenotype is also direct in GEP.

◎ For example, consider you have the following K-expression:

Q*+*a*Qaaba

◎ This can be mapped to the tree below by start filling the tree at a level-by-level fashion, by respecting the number of arguments for each function processed:



GEP encoding advantages

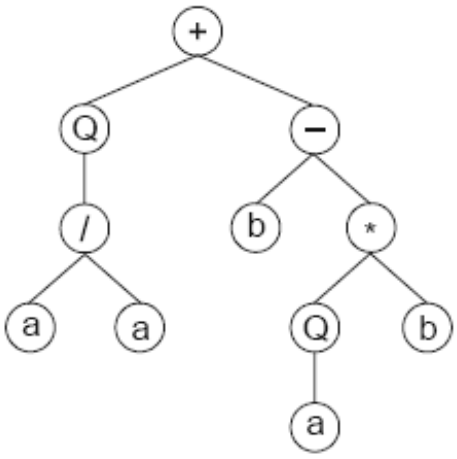
⌘ Although both GP and GEP use program trees, GEP has certain advantages:

- ◎ GEP introduces fixed length linear chromosomes.
- ◎ What varies is not the length of chromosomes (which is constant), but the length of the ORFs.
  - length (ORF) ≤ length (chromosome).
- ◎ It creates non-coding regions, which allow modification of the genome using any genetic operator without restriction, always producing correct offspring, without the need for handling invalid offspring.
- ◎ GEP genes are composed of a Head (elements from F∪T) and a Tail (elements from T). This allows different alphabets in different regions of each string.

Example:
For head-length h = 11,
tail-length t = 4,
and alphabet {Q,*,/,+,-, a,b},
we could have (tail in bold):

+ Q - / b * a a Q b a **a b b a**

So, the termination occurs in the 11th position, but the string has 15 genes !!!



Mutation

- Mutation is applied in specific genes as in discrete GAs.
- A random gene is selected for mutation with probability p_m.
- However, the chromosome structural organisation is preserved.
  - That is, a head element is mutated to with another from F ∪T, while a tail element with one from T.

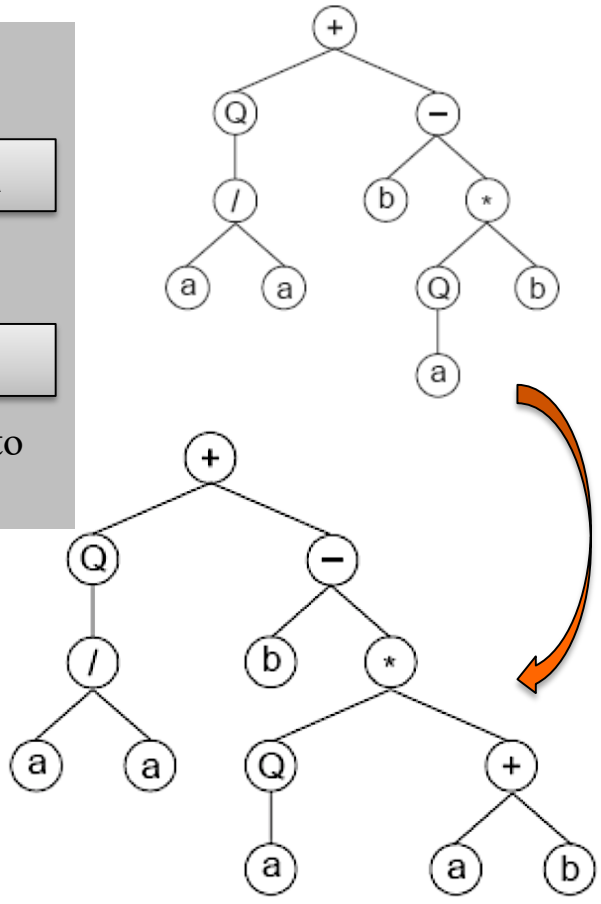
Example:
The chromosome:

+ Q - / b * a a Q b a	a b b a

has its "b" changed to "+".
This produces:

+ Q - / b * a a Q + a a b	b a

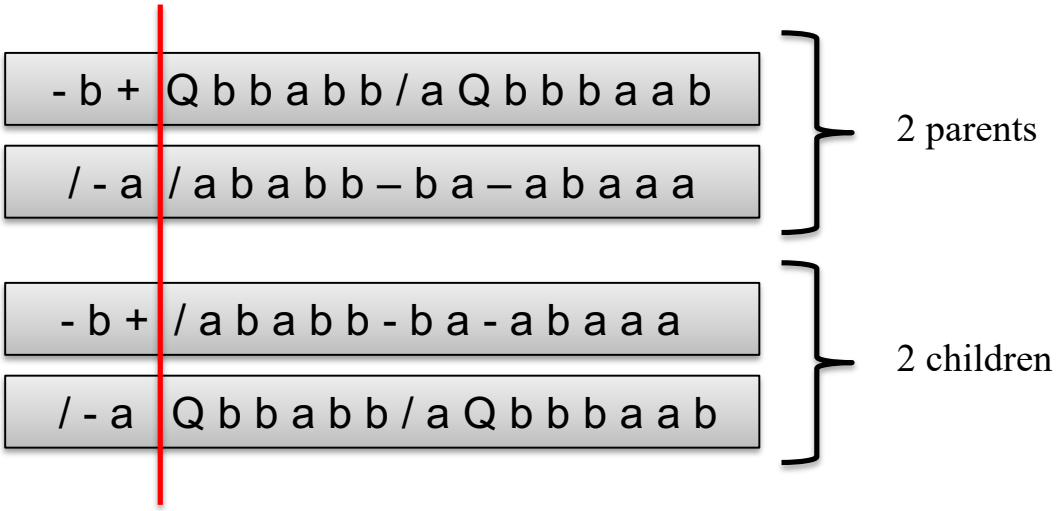
which shifts termination 2 positions to the right (tail is of length 2 now).



Crossover

⌘ In its simplest implementation, the current linear encoding allows a 1-point crossover.

◎ An example of this is the following:



⌘ The problem here is that offspring may exhibit completely different structure and properties from those of the parents; this can be good for diversity but also disrupt the parental schemata.

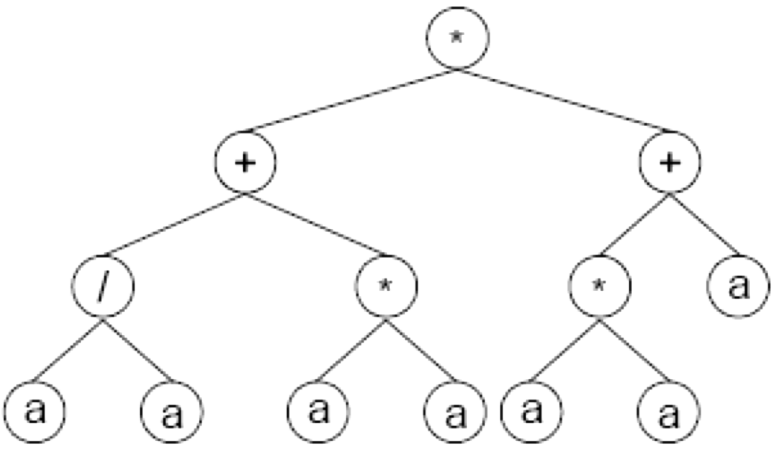
Example - applications

⌘ **Symbolic regression**:

- Data samples are taken from the (unknown) function:

  $f(a)=a^4+a^3+a^2+a$

- F = {+, -, *, /} (surprisingly, you are not allowed to use “power” operation!)
- T = {a}
- Solution produced: `* + + / * * a a a a a a a a`
- This corresponds to: $(1+a^2)*(a+a^2) = a+a^2+a^3+a^4$
- The tree phenotype is:



<END of Chapter 5>

Extras:
practice & examples

Exercise

All of the following examples use the basic function set: +, −, * (multiply), % (protected divide), <, >, and the terminal set: 0, 1, 2, …15, *X* (an input), *Y* (an input).

1. The cosine function can be approximated by,

$$\cos(X) \approx 1 - \frac{X^2}{2} + \frac{X^4}{24}$$

Draw a tree structure that would recreate this function.

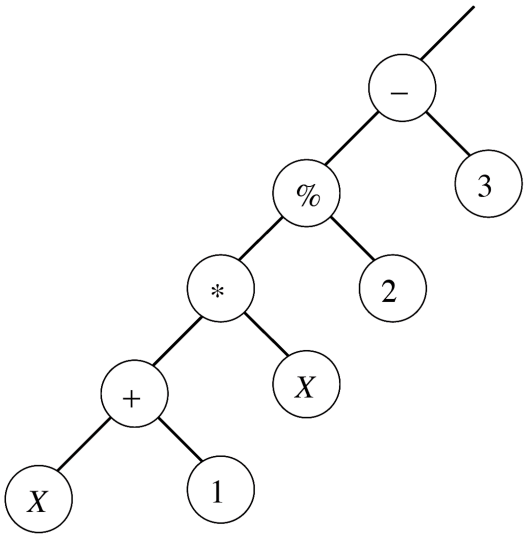
What is the complexity of the tree, in terms of basic functions used?

2. Draw a tree structure that would recreate the function,

$$f(X,Y) = \begin{cases} 4X & X > Y \\ 2Y & X \le Y \end{cases}$$

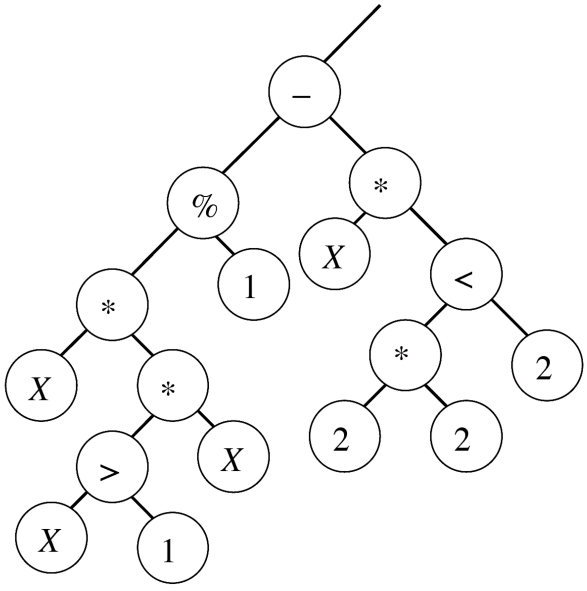
What is the complexity of the tree, in terms of the total number of nodes used?

3. Write down the function that corresponds to the tree structure given below,



What is the complexity of the tree, in terms of the total number of nodes used?

4. Write down the function that corresponds to the tree structure given below,



What is the complexity of the tree, in terms of the number of basic functions used?

Which branch (or branches) of this tree is an intron?